

MyTask Space

Personal Productivity Web Application

Project Type	Full-Stack Web Application
Tech Stack	Python (Flask) · SQLite · HTML · CSS · JavaScript
Deployment	Railway (Cloud Platform)
Date	April 2026
Built by	MANEESHA P.C.

1. Introduction

MyTask Space is a full-stack personal productivity web application built entirely from scratch. It gives each registered user a private, cloud-saved workspace to organise their day with a daily timetable, weekly planner, sticky notes and a to-do list — all in one clean interface with multiple theme options.

This project was conceived, designed and built using prompt engineering as a learning tool — exploring how to think through software architecture, write backend Python code, design CSS layouts and connect a frontend to a REST API. Every line of code represents a hands-on learning journey.

1.1 Why This Project?

The motivation was to build something genuinely useful while learning the complete web development lifecycle — from designing a database schema, writing server-side logic, building a polished UI, handling user authentication and deploying to a live cloud server. MyTask Space is the result.

1.2 Core Objectives

- Build a secure multi-user system where each user's data is completely private
- Create a beautiful, functional productivity tool with a warm aesthetic
- Learn Flask, SQLite and REST API design by building something real
- Deploy a live production application on Railway cloud platform
- Practise prompt engineering to solve real coding challenges

2. Features Overview

MyTask Space includes six core modules, all data persisted in a per-user SQLite database and synced instantly via a REST API.



Secure Authentication

Full register / login / logout flow. Passwords are hashed using Werkzeug's PBKDF2 — never stored as plain text. Sessions are managed with Flask-Login. Each user's data is completely isolated.



Daily Timetable

Add time-slot entries with start and end times, task description and a colour label. Rows are sorted by start time automatically. Edit or delete any row via a context menu.



Weekly Planner

A 7-column Monday–Sunday grid. Add, edit or delete tasks for any day. Tasks show their time range and are colour-coded. The planner opens as a full-screen modal overlay.



Sticky Notes

Up to 8 sticky notes per user. Notes are automatically placed — 4 on the left panel and 4 on the right. Each note can be edited or deleted via a dots menu.



To-Do List

A floating to-do panel with persistent checklist items. Items can be checked off, edited inline, or deleted. Changes are saved explicitly with a 'Save' button.



Background Themes

Six theme options: White, Black, Light Yellow, Light Pink, Light Blue, Light Green. The selected theme is saved to the database and restored on next login.



PDF Export

One-click export of the Daily Planner or Weekly Planner as a print-ready PDF, styled with the app's fonts and colours. Opens in a new browser window for printing.



Account Deletion

Users can permanently delete their account from the settings menu. Requires email + password confirmation. All user data is removed via SQLite CASCADE DELETE.

3. Application Screenshots

3.1 Login Page

The login screen uses a warm cream-coloured card design with Playfair Display typography, gold accents and Dancing Script for the subtitle — giving it a premium personal-planner feel.

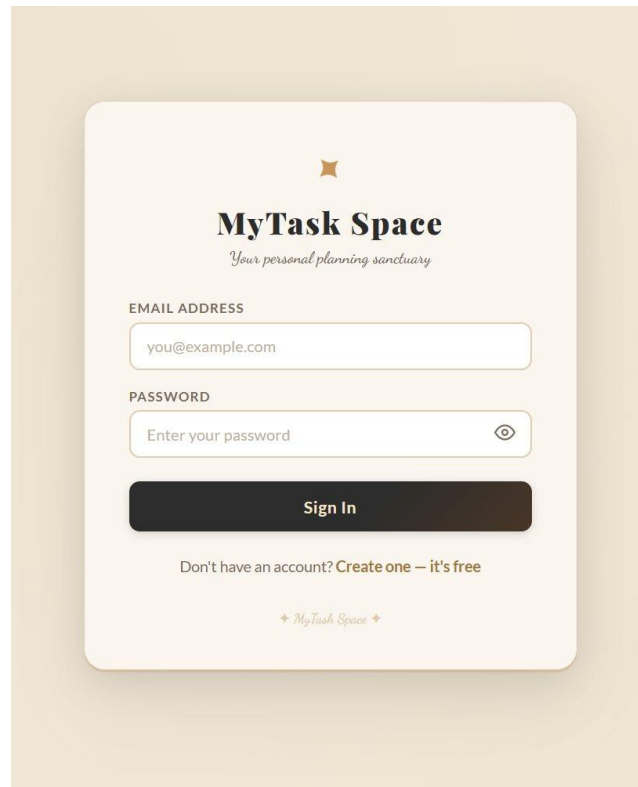


Figure 1: Login page — MyTask Space

3.2 Daily Timetable

The main workspace showing the daily timetable with multiple entries, time ranges displayed, colour-coded task names and the left toolbar with all quick-access buttons. The To-Do list panel is visible floating in the bottom-right corner.

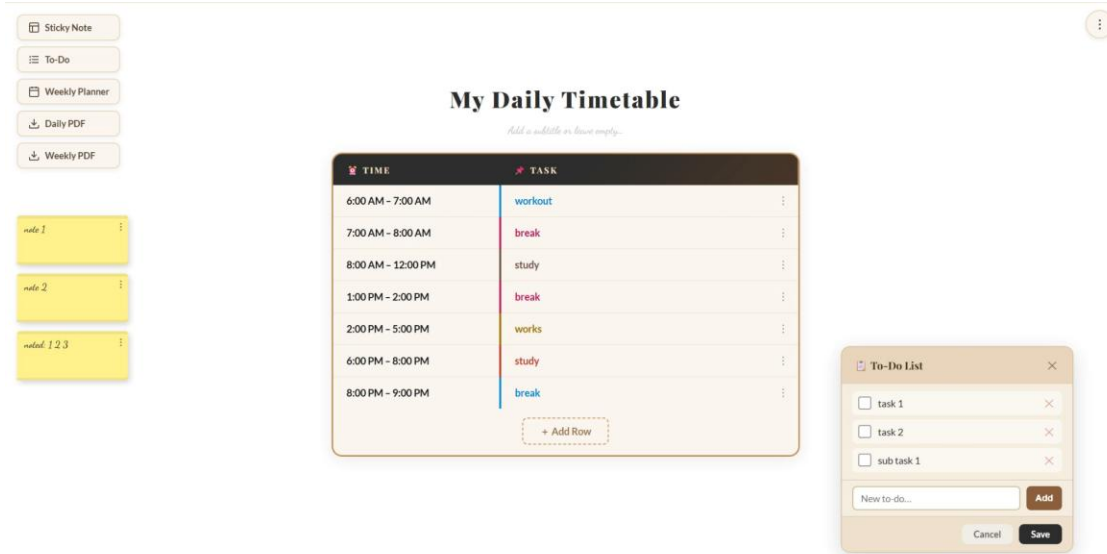


Figure 2: Main workspace — White theme with To-Do panel open

The same workspace with the Light Blue background theme applied. The 3-dot settings menu is open on the right, showing the theme picker and the Delete Account option under the Account section.

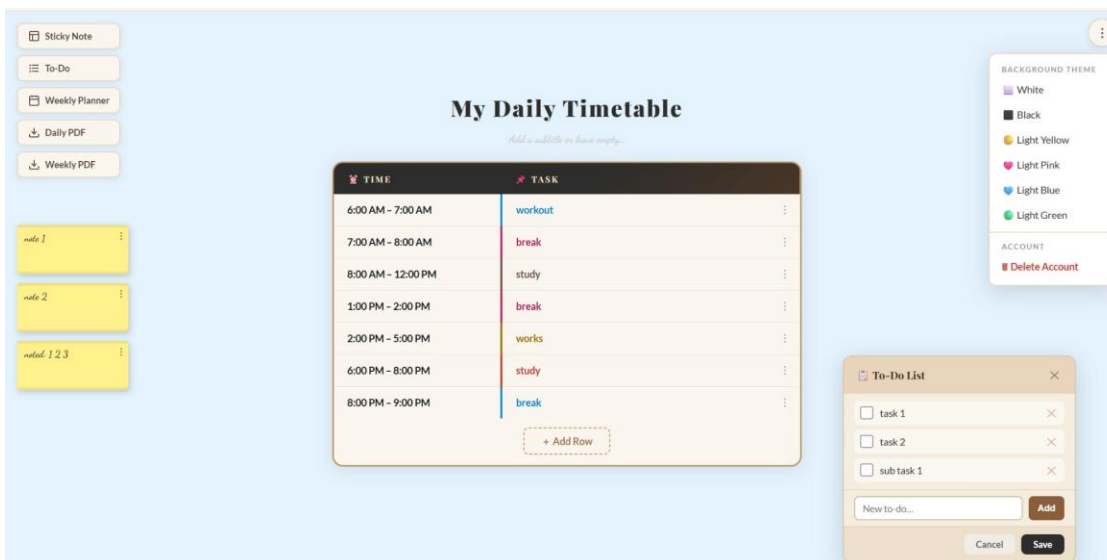


Figure 3: Main workspace — Light Blue theme with settings menu open

3.3 Add Row Modal

The 'Add Row' modal for the daily timetable, showing the custom AM/PM time picker for both Start and End time, the task text field, and the colour-picker with 10 colour options.

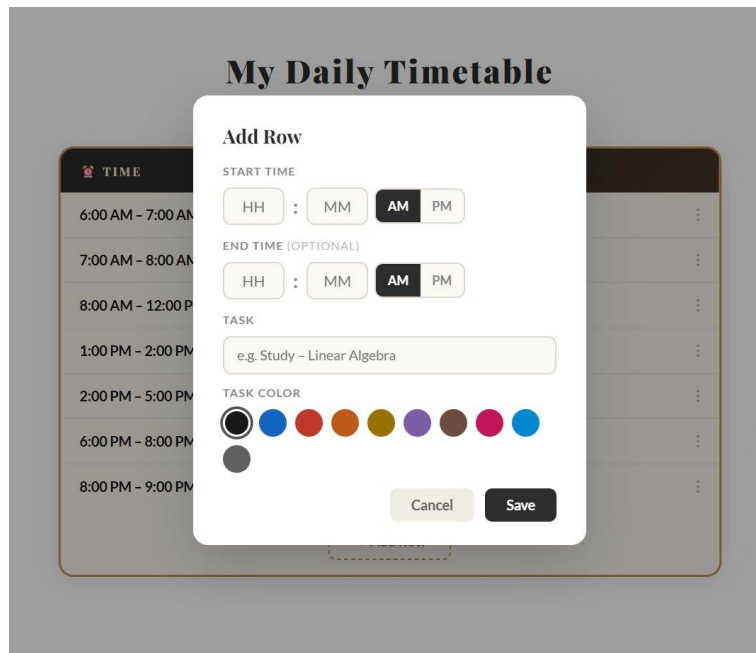


Figure 4: Add Row modal — custom time picker and colour selector

3.4 Weekly Timetable Planner

The full-screen weekly planner showing all 7 days in a grid. Each day column shows tasks with their time range and colour-coded labels. The bottom of each column has a '+ Add' button. The sticky notes panel is visible on the left side behind the modal.

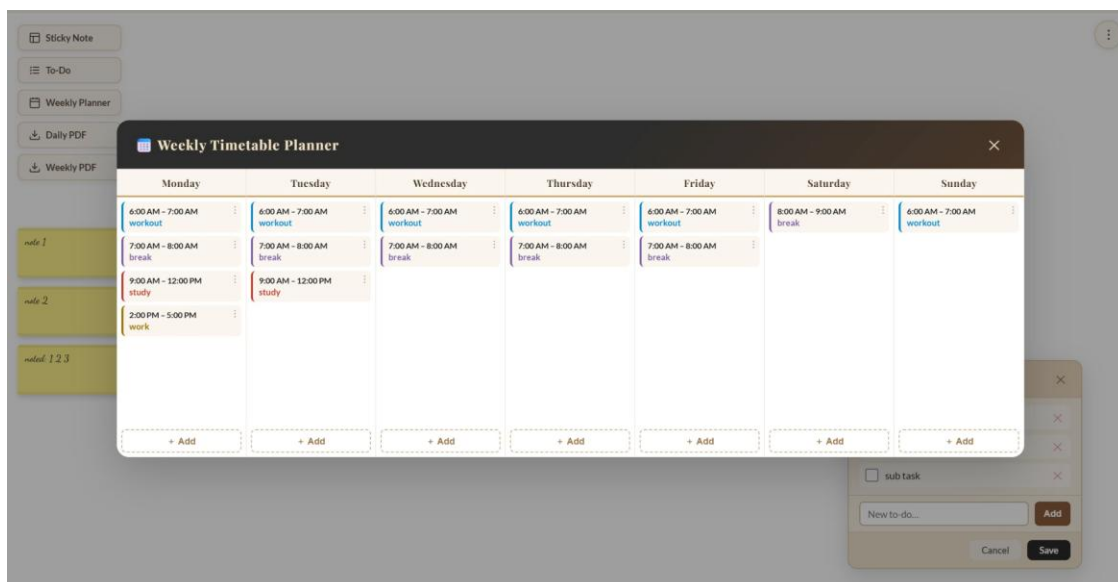


Figure 5: Weekly Timetable Planner — full-screen grid view

4. System Architecture

MyTask Space follows a classic three-tier architecture: a browser-based frontend, a Python/Flask backend, and a SQLite database. All tiers communicate via a REST API using JSON.

 Browser (HTML / CSS / JS)	 Flask Routes (app.py)	 SQLite Database (data.db)
↓ HTTP Request	↓ Query / Write	↓ Returns rows
app.js Fetch API calls	Python Logic Flask-Login + Auth	Tables: users / rows / notes todos / weekly

The browser never directly touches the database. All data access goes through Flask route handlers that verify the user's session, then query or modify the SQLite database, and return a JSON response. The JavaScript frontend uses the Fetch API to communicate with these routes asynchronously — no full page reloads after the initial load.

5. Technology Stack

Layer	Technology	Purpose
Backend	Python 3 + Flask	Web framework - handles routing, sessions and API endpoints
Auth	Flask-Login + Werkzeug	Session management and PBKDF2 password hashing
Database	SQLite	Lightweight file-based relational database (data.db)
Frontend	HTML5 + CSS3	Semantic structure and all visual design / theming
Frontend	Vanilla JavaScript	Dynamic UI - modals, Fetch API calls, AM/PM picker
Fonts	Google Fonts	Playfair Display (headings) · Lato (body) · Dancing Script (accents)
Deployment	Railway	Cloud hosting with auto-deploy from GitHub on every push
Server	Gunicorn	Production WSGI server (replaces Flask dev server in production)

6. Database Schema

All data is stored in a single SQLite file (data.db). The schema is created automatically on first launch using CREATE TABLE IF NOT EXISTS statements. There are 6 tables, all linked to the users table via foreign keys with CASCADE DELETE — so deleting a user wipes all their data cleanly.

Table	Key Columns	Purpose
users	id, email, password_hash	Stores registered accounts with hashed passwords
settings	user_id, key, value	Per-user preferences: heading, theme, subtitle
timetable_rows	time_start, time_end, task, task_color	Daily planner entries with colour coding
sticky_notes	content, position (left/right)	Up to 8 notes per user, auto-positioned
todos	content, done, sort_order	Checklist items with completion state
weekly_tasks	day, time_start, time_end, task	Weekly planner entries per day-of-week

6.1 Key Design Decisions

- CASCADE DELETE — all user data tables reference users(id) ON DELETE CASCADE. Deleting a user account removes everything automatically.
- PRAGMA foreign_keys = ON — explicitly enabled per connection, as SQLite does not enforce foreign keys by default.
- Settings as key-value pairs — instead of fixed columns, settings are stored as (user_id, key, value) rows. This makes it easy to add new settings without changing the schema.
- password_hash only — raw passwords are never stored. Werkzeug's generate_password_hash uses PBKDF2-SHA256 with salting.

7. REST API Reference

All API endpoints require an active login session (enforced with the @login_required decorator). Unauthenticated requests are redirected to the login page. All request and response bodies use JSON.

Method	Endpoint	Description
GET	/api/settings	Fetch user's saved preferences
POST	/api/settings	Save theme / heading / subtitle
GET	/api/rows	Load all daily timetable rows
POST	/api/rows	Add a new timetable row
PUT	/api/rows/:id	Edit an existing timetable row
DELETE	/api/rows/:id	Delete a timetable row
GET	/api/notes	Load all sticky notes
POST	/api/notes	Create a new sticky note (max 8)
PUT	/api/notes/:id	Edit sticky note content
DELETE	/api/notes/:id	Delete a sticky note
GET	/api/todos	Load all to-do items
POST	/api/todos	Add a new to-do item
PUT	/api/todos/:id	Update content / done state
DELETE	/api/todos/:id	Remove a to-do item
GET	/api/weekly	Load all weekly planner tasks
POST	/api/weekly	Add weekly task for a day
PUT	/api/weekly/:id	Edit weekly task
DELETE	/api/weekly/:id	Delete weekly task
POST	/api/delete-account	Verify credentials and delete account + all data

8. Frontend Design

8.1 Design Philosophy

The UI was designed around a warm, paper-like aesthetic - cream backgrounds, gold accents, and serif headings, to feel like a physical planner rather than a generic productivity app. Three Google Fonts were carefully chosen:

- Playfair Display — headings and table headers (editorial, classic)
- Lato — body text and UI labels (clean, readable)
- Dancing Script — subtitles and decorative flourishes (personal, warm)

8.2 CSS Variables & Theming

All colours and fonts are defined as CSS custom properties (variables) in root, making the theme system easy to maintain. Six body class themes (theme-white, theme-black, etc.) switch the background with a single class change applied by JavaScript, then saved to the database.

8.3 Mobile Responsiveness

The app is fully responsive. On screens narrower than 700px:

- The left toolbar collapses off-screen and is toggled by a hamburger button
- The timetable table fills the full width
- Modals become full-width with reduced padding
- The To-Do window stretches full width at the bottom of the screen
- The weekly planner uses horizontal scrolling for the 7-column grid

9. Security Implementation

9.1 Password Security

Werkzeug's `generate_password_hash` function is used, which applies PBKDF2-SHA256 with a random salt. Passwords are never stored, logged, or transmitted in plain text. `check_password_hash` is used for verification at login.

9.2 Session Management

Flask-Login manages user sessions with a signed cookie. The `SECRET_KEY` used to sign cookies is read from an environment variable on Railway, or auto-generated and stored in a `.secret_key` file for local development, ensuring sessions survive server restarts.

9.3 Data Isolation

Every database query includes `AND user_id=?` as a filter condition. This ensures that even if a user somehow guessed another user's row ID, they cannot read or modify that user's data.

9.4 SQL Injection Prevention

All database queries use parameterised statements. User input is never concatenated into query strings.

9.5 Account Deletion

The delete-account endpoint re-verifies both the user's email and password before proceeding. This prevents CSRF-style attacks where a malicious page could trigger deletion. The backend returns a 400 error if credentials don't match, before touching any data.

10. Deployment (Railway)

The application is deployed on Railway, a modern cloud platform that auto-deploys from a connected GitHub repository on every push.

Platform	Railway (railway.app)
Trigger	Automatic on every git push to main branch
Server	Gunicorn WSGI server (defined in Procfile)
Procfile	web: gunicorn app:app
Database	SQLite file (data.db) — lives on Railway's ephemeral filesystem

12. Future Improvements

The following improvements are planned or would be natural next steps for the project:

- PostgreSQL integration — replace SQLite with a persistent database for production use on Railway
- Dark mode auto-detection — detect the user's system preference with prefers-color-scheme media query
- Drag-and-drop reordering — allow rows and to-do items to be reordered by dragging
- Reminders / notifications — push notification or email reminders for timetable entries
- Export to Google Calendar — allow weekly tasks to be exported as .ics files
- Mobile app — wrap the web app in a PWA (Progressive Web App) for mobile home screen installation
- Shared workspaces — allow users to optionally share a planner with another user
- CSRF protection — add Flask-WTF CSRF tokens to all POST forms and API calls

13. Conclusion

MyTask Space is a complete, fully functional full-stack web application built from scratch as both a useful productivity tool and a learning project. It demonstrates a solid understanding of the full web development lifecycle:

- Designing a multi-user backend with secure authentication
- Building a REST API with proper data isolation
- Creating a polished, responsive frontend with a strong visual identity
- Deploying a live application to the cloud

What makes this project meaningful is not just the final result, but the process .